Escape the Geese – Requirements

Cohort 2 Team 3 – ‘Kostas Barmpis and Dimitris Kolovos’

Team members:

Sruthiksha Ananth

Louis Price

Sarah James

Ben Smejko

Henrik Bowden

Tim Giles

June Njiru

(a) Introduction: Elicitation, Negotiation, and Presentation

Elicitation

We began by reviewing the customer's product brief and the assessment paper. The brief set out the core game but left several requirements unclear or missing, particularly quality requirements such as performance, supported platforms and visual style. We prepared targeted questions grouped by topic (platform and presentation, gameplay, items and events, accessibility, scope) and asked them at a customer meeting on 9 October 2026, for example "What hardware should this run on, and does it need to support mobile devices?" The customer confirmed a non-violent, top-down 2D desktop game for Windows, macOS and Linux that loads within a couple of seconds, introduces prospective students to the University of York and its geese, and conveys important information through at least two of sound, visuals and text. The full questions and answers are in our meeting notes on the team website Customer Meeting notes where each agreed point is mapped to the requirement IDs below.

Negotiation

Several requirements were agreed through discussion rather than simply recorded. The choice of 2D over 3D balanced the customer's preference against what was achievable within the project timeline. The customer wanted the geese to feel threatening without violence, so we agreed an exhaustion meter that works like health but is presented as the player's tiredness (FR_EXHAUSTION_METER). We proposed saving progress and a leaderboard; the customer declined both, so they are recorded as out of scope. Difficulty levels and an alternative escape win were left optional (May) and may be revisited in Assessment 2. Open questions will be raised at future meetings, and changes will be tracked by requirement ID.

Presentation

Following Sommerville [1], we separate user requirements, written in plain language so the customer can check them, from system requirements, which state precisely what the system must do so the team can build and test against them. System requirements are divided into functional requirements (FR_), describing what the game does, and quality requirements (QR_), describing how well it must do it [1]. As in the Volere template [2], constraints (CR_) are recorded separately because they are fixed conditions on the solution, such as platform and programming language, rather than features, and each records its source. Every requirement has a unique, descriptive identifier and a priority using the keywords of ISO/IEC/IEEE 29148 [3]: Shall (mandatory), Should (desirable) and May (optional, dropped first if time runs short). Each system requirement links to the user requirement it satisfies, supporting traceability through design, implementation and testing [3]. Functional requirements are written so that a single test can show whether they are met, and constraints and quality requirements include a measurable fit criterion [2], because statements such as "the game should start quickly" cannot be tested without one [1]. Tables keep the requirements easy to scan for both the customer and the team.

(b) User and System Requirements

Priority keywords follow ISO/IEC/IEEE 29148: Shall = mandatory, Should = desirable, May = optional. Sources Q1 to Q16 refer to the questions in our customer meeting notes .

User Requirements

ID	Description	Priority	Source
UR_MOVEMENT	The user shall be able to control their character on the campus map	Shall	Brief
UR_GEESE	Geese shall be the main obstacle to the user completing the game	Shall	Brief, Q8
UR_ITEMS	The user shall have items they can use to disperse or distract geese.	Shall	Brief, Q11
UR_EVENTS	The user shall encounter events that help or hinder them.	Shall	Brief, Meeting
UR_GAME_END	The game shall end in a clear win or loss.	Shall	Q9, Q10
UR_SCORE	The user shall receive a score that rewards good item use and timely progress. No leaderboard or saved scores.	Shall	Brief, Q15
UR_TUTORIAL	A first-time user shall be able to learn the controls and goal inside the game	Shall	Q16
UR_ACCESSIBILITY	The game shall be accessible to players of all skills and abilities.	Shall	Q6, Q14
UR_RELATE	The game should be recognisable to York students and applicants through landmarks and references.	Should	Q3, Q8, Q13
UR_PERFORMANCE	The game should start quickly and run smoothly so players stay engaged.	Should	Q5
UR_EXTENSIBLE	New items, events and features should be addable in later iterations without a redesign	Should	Q16

Constraints

ID	Description	Source	Fit Criterion
CR_PLATFORMS	The game shall run on Windows, macOS and Linux desktops.	Q1	A full playthrough completes on each OS with no crash
CR_HARDWARE	The game shall run on standard desktop or laptop without dedicated graphics	Q1, Q5	Full playthrough without crashing on a 4-core CPU, 8 GB RAM and integrated graphics
CR_JAVA	The game shall be written in Java 17	Assessment Paper	Build targets Java 17; compiles and runs on JDK 17.
CR_ENGINE	The game shall use a stable, well documented 2D Java framework that is widely used for similar games.	Q4	Framework has public documentation and a release in the last 12 months
CR_VIEW	The game shall use a top-down 2D view	Q2	Walking through every screen shows a 2D top-down view
CR_JAR	The game shall be delivered as one executable JAR that includes all dependencies and needs no configuration.	Assessment paper, Q15	The JAR launches with java -jar on the minimum-spec machine with nothing else installed except Java.

CR_LICENCE	Third-party assets and libraries shall be licenced compatibly and attributed.	Brief, Q4	The asset log records source, licence and for 100% of third-party assets and libraries
CR_PEACE	The game shall not show geese or the player being harmed and shall show no health bar	Q7, Q8	Review of every sprite, animation and HUD element finds no injury, death, blood or health bar.

Out of scope: Saving progress, high scores, leaderboard or player data (Q15); mobile platforms (Q1); more than one map or entering buildings (Q3).

Functional Requirements

ID	Description	Priority	User Requirement
Movement and Map			
FR_MOVE_KEYS	The player shall move up, down, left and right using both WASD and the arrow keys.	Shall	UR_MOVEMENT
FR_MAP_BOUNDS	The player shall not be able to move outside the map boundary	Shall	UR_MOVEMENT
FR_LANDMARKS	The map shall include at least 2 recognisable York landmarks (e.g. Central Hall, the Piazza)	Should	UR_RELATE
FR_INTERACTABLES	Interactable objects shall be marked by a non-colour cue, such as sparkle, as well as colour	Shall	UR_ACCESSIBILITY
FR_CHAR_LOOK	The player character should be identifiable as a York undergraduate (e.g. York hoodie)	Should	UR_RELATE
Geese			
FR_GEESE_BLOCK	Geese shall block the player's movement	Shall	UR_GEESE
FR_GEESE_WAVES	Geese shall arrive in waves	Shall	UR_GEESE
FR_WAVE_COUNTDOWN	A countdown of about 10 seconds should be shown between waves	Should	UR_GEESE
FR_EXHAUSTION_METER	An exhaustion meter shall fill while geese are close to or blocking the player in place of health	Shall	UR_GEESE UR_GAME_END
Items			
FR_START_ITEMS	The player shall start with 5 inventory items.	Shall	UR_ITEMS
FR_MAP_ITEMS	At least 5 pickup items shall be placed on the map.	Shall	UR_ITEMS
FR_ITEM_ICON	The inventory shall show each item as an icon.	Shall	UR_ITEMS
FR_ITEM_DISPERSE	Using an item shall disperse or distract geese within its range without harming them.	Shall	UR_ITEMS UR_GEESE
FR_ITEM_TYPES	At least 2 item types shall differ in range, strength or target (e.g. area food, a thrown item)	Shall	UR_ITEMS
FR_ITEM_MISUSE	Throwing a distraction item straight at a goose should enrage it, making it faster for a short time.	Should	UR_ITEMS
Events			
FR_EVENTS_COUNT	At least 5 triggerable events shall exist.	Shall	UR_EVENTS
FR_EVENT_AUDIO	Each event shall play a distinct sound when triggered.	Shall	UR_ACCESSIBILITY
FR_EVENT_VISUAL	Each event shall show a visual cue of its cause and effect, such as speech bubble.	Shall	UR_ACCESSIBILITY
FR_EVENT_ITEMS	Events should be able to add or remove items on the map.	Should	UR_EVENTS
Scoring and HUD			
FR_SCORE_ITEMS	The score shall increase when an item disperses geese.	Shall	UR_SCORE
FR_SCORE_SPEED	Geese dispersed sooner after a wave begins shall earn more points.	Shall	UR_SCORE
FR_TIMER	A 5-minute countdown shall be visible throughout gameplay.	Shall	UR_GAME_END
FR_HUD_DISPLAY	The HUD shall show the timer, score, inventory and exhaustion meter throughout gameplay.	Shall	UR_SCORE UR_ITEMS
FR_USAGE_COUNTER	The game shall display the number of items used and events triggered.	Shall	UR_EVENTS UR_ITEMS

Game Flow			
FR_START_PAUSED	The game shall start paused.	Shall	UR_ACCESSIBILITY
FR_TUTORIAL	A tutorial screen showing the controls, items and goal shall be reachable from main menu.	Shall	UR_TUTORIAL
FR_WIN	The game shall end in a win when the player clears every wave before the timer ends.	Shall	UR_GAME_END
FR_LOSE	The game shall end in a loss, with the character returning to their room, when the timer ends or the exhaustion meter fills.	Shall	UR_GAME_END
FR_ESCAPE_WIN	A rare, hard-to-find escape route may give an alternative win	May	UR_GAME_END
FR_DIFFICULTY	The player may choose difficulty that changes goose speed, count or wave timing.	May	UR_ACCESSIBILITY
Settings			
FR_PAUSE	The player shall be able to pause at any time; the timer stops while paused.	Shall	UR_ACCESSIBILITY
FR_PAUSE_MENU	Pausing shall open a menu with volume and full screen settings.	Shall	UR_ACCESSIBILITY
FR_FULLSCREEN	The player shall be able to toggle full screen.	Shall	UR_ACCESSIBILITY
FR_VOLUME	The player shall be able to raise or lower the volume.	Shall	UR_ACCESSIBILITY

Quality Requirements

ID	Description	Priority	User req.	Fit Criterion
QR_STARTUP	The game should start quickly	Should	UR_PERFORMANCE	Main menu appears within 3s of launching the JAR in 5 consecutive launches on the minimum-spec machine.
QR_FRAME_RATE	Gameplay should run smoothy.	Should	UR_PERFORMANCE	At least 30 fps for a full 5-minute session with the maximum number of geese, on the minimum-spec machine.
QR_OPERABILITY	A first-time player should understand the premise and play with outside help.	Should	UR_TUTORIAL	In user-testing, at least 4 of 5 first-time players start the game, use an item and trigger an event unaided.
QR_WINNABILITY	Most players should be able to win.	Should	UR_ACCESSIBILTY	At least 70% of test players win on their first attempt
QR_HUD_VISIBILITY	The timer, score, inventory and meter should stay readable in all display modes.	Should	UR_ACCESSIBILITY	At 1280x720 and 1920x1080, windowed and full screen, no HUD element overlaps another.
QR_EXTENSIBILITY	The game should be easy to extend between iterations.	Should	UR_EXTENSIBLE	Adding a new item or event needs only a new class and asset, with no change to Player, Goose or GameScreen.

References

[1] I. Sommerville, Software Engineering, 10th ed. Harlow, UK: Pearson, 2016.

[2] S. Robertson and J. Robertson, Mastering the Requirements Process: Getting Requirements Right, 3rd ed. Upper Saddle River, NJ: Addison-Wesley, 2012.

[3] Systems and Software Engineering — Life Cycle Processes — Requirements Engineering, ISO/IEC/IEEE 29148:2018, 2018